

C2SafeRust-LLM

Module 3: Per-Function Semantic Analysis -- Line-by-Line Explanation

For Beginners | Mini Project S3

Module 3: Per-Function Semantic Analysis

Line-by-Line Code Explanation for Beginners

Files: `src/module3_semantic/mod.rs`, `base_analyser.rs`, `pointer_chain.rs`, `union_transformer.rs`

What does Module 3 do? Module 3 takes the AST produced by Module 2 and analyzes EACH FUNCTION INDIVIDUALLY. For every function, it asks three questions: 1. What variables exist in this function, and how are they typed? (Base Analysis) 2. How deep are the pointer chains? Is it `int`, `int*`, `int**`? (Pointer Chain Analysis) 3. Does this function use C unions? Can we determine which field is active? (Union Transformer) The results are stored in the D1 SemanticModel for each function. This D1 data feeds directly into Module 4 (interprocedural analysis) and Module 5 (code generation).

```
TranslationUnitAST (from Module 2)
|
| For EACH function:
v
analyze_function_base()    --> symbol_table, def_use_chains, memory_patterns
analyze_pointer_chains()   --> pointer_depth_classifications
transform_unions_per_function() --> local_union_transforms, unresolved_union_cases
|
v
HashMap<String, SemanticModel> (D1: one entry per function name)
```

File 1: `src/module3_semantic/mod.rs`

(The Orchestrator — Runs All Three Analyses Per Function)

Lines 1-7: Module declarations and re-exports

```
pub mod base_analyser;
pub mod pointer_chain;
pub mod union_transformer;

pub use base_analyser::analyze_function_base;
pub use pointer_chain::analyze_pointer_chains;
pub use union_transformer::transform_unions_per_function;
```

Code	Explanation
<code>`pub mod base_analyser;`</code>	Declares the <code>`base_analyser.rs`</code> sub-module and makes it public.
<code>`pub mod pointer_chain;`</code>	Declares the <code>`pointer_chain.rs`</code> sub-module.
<code>`pub mod union_transformer;`</code>	Declares the <code>`union_transformer.rs`</code> sub-module.
<code>`pub use base_analyser::analyze_function_base;`</code>	Re-exports the main function from <code>`base_analyser.rs`</code> so users can call <code>`module3_semantic::analyze_function_base()`</code> directly instead of the longer path.

Lines 9-11: Cross-module imports

```
use std::collections::HashMap;
use crate::module2_ast::TranslationUnitAST;
use crate::data_stores::semantic_models::SemanticModel;
```

Import	Why needed
<code>`HashMap`</code>	The return type is <code>`HashMap`</code> — one entry per function name.
<code>`TranslationUnitAST`</code>	The input — the whole-program AST from Module 2.
<code>`SemanticModel`</code>	The D1 output data structure from <code>`data_stores/semantic_models.rs`</code> .

Lines 13-33: The Main Entry Point

```
pub fn analyze_translation_unit_semantics(
    tu_ast: &TranslationUnitAST,
) -> HashMap<String, SemanticModel> {
    let mut semantic_models = HashMap::new();

    for func in &tu_ast.functions {
        // Step 3.1: Base Semantic Analysis
        let mut model = analyze_function_base(func);

        // Step 3.2: Pointer Chain Analysis
        analyze_pointer_chains(func, &mut model);

        // Step 3.3: Union-to-Enum Transformation
        transform_unions_per_function(func, tu_ast, &mut model);

        semantic_models.insert(func.name.clone(), model);
    }

    semantic_models
}
```

Line	Code	Explanation
<code>`tu_ast: &TranslationUnitAST;`</code>	Borrow the AST	Takes a reference — doesn't take ownership. Module 2's data stays valid.
<code>`-> HashMap`</code>	Return type	Returns a dictionary: function name → its semantic model.
<code>`let mut semantic_models = HashMap::new();`</code>	Empty dictionary	Will be filled with one entry per function.
<code>`for func in &tu_ast.functions`</code>	Loop over functions	Iterates over every <code>`FunctionAST`</code> in the parsed program. <code>`&`</code> means we borrow each function, not move it.
<code>`let mut model = analyze_function_base(func)`</code>	Step 3.1	Creates a fresh <code>`SemanticModel`</code> and fills in the symbol table, def-use chains, and memory patterns. <code>`mut`</code> because steps 3.2 and 3.3 will modify it.
<code>`analyze_pointer_chains(func, &mut model)`</code>	Step 3.2	Adds pointer depth classifications to the SAME model. Takes <code>`&mut model`</code> — a mutable reference — so it can modify the model in place.
<code>`transform_unions_per_function(func, tu_ast, &mut model)`</code>	Step 3.3	Also modifies the same model. Needs <code>`tu_ast`</code> to look up union definitions in the whole program.
<code>`semantic_models.insert(func.name.clone(), model)`</code>	Store result	Maps <code>`"swap"`</code> → the completed <code>`SemanticModel`</code> for <code>`swap`</code> . <code>`clone()`</code> makes a copy of the name string since <code>`insert`</code> takes ownership.

Design Pattern — Build, Augment, Store: Notice how `model` is created by `analyze_function_base()`, then MODIFIED in place by `analyze_pointer_chains()` and `transform_unions_per_function()`. This is Rust's way of building a data structure incrementally while maintaining clear ownership:

`analyze_function_base` owns and returns `model`, then the next calls borrow it mutably to add more data.

File 2: `src/module3_semantic/base_analyser.rs`

(Symbol Tables, Def-Use Chains, Memory Patterns)

Purpose: The "base" analysis does the fundamental work: building a symbol table (what variables exist?) and tracking how each variable is used (read? written? malloc'd? freed?). This is the data that most directly drives the ownership inference in Module 5.

Lines 1-2: Imports

```
use crate::module2_ast::ast_parser::FunctionAST;
use crate::data_stores::semantic_models::{MemoryPattern, SemanticModel, UsePoint};
```

Import	Why needed
<code>`FunctionAST`</code>	The input — one function from the AST.
<code>`MemoryPattern`</code>	The enum (<code>`Read`</code> , <code>`Write`</code> , <code>`Allocate`</code> , <code>`Free`</code>) stored in the model.
<code>`SemanticModel`</code>	The D1 model we're building.
<code>`UsePoint`</code>	Stores <code>`(line, column)`</code> locations for def-use chains.

Lines 4-57: The Main Function `analyze_function_base()`

```
pub fn analyze_function_base(func: &FunctionAST) -> SemanticModel {
    let mut model = SemanticModel::new(func.name.clone());
    // ...
    model
}
```

Code	Explanation
<code>`func: &FunctionAST;`</code>	Borrows the function AST node.
<code>`-> SemanticModel`</code>	Returns a fully populated semantic model. No <code>`Result`</code> — this analysis always succeeds (even if the function is empty).
<code>`SemanticModel::new(func.name.clone())`</code>	Creates a fresh model with all HashMaps and Vecs empty. The function's name is cloned (copied) into the model.

Step 1: Build the Symbol Table from Parameters (Lines 9-13)

```

for param in &func.parameters {
    let type_desc = format!("{}", param.c_type, "{}".repeat(param.pointer_depth));
    model.symbol_table.insert(param.name.clone(), type_desc);
}

```

Line	Code	Example with <code>int **a</code>
<code>`for param in &func.parameters;`</code>	Loop over parameters	Iterates each <code>ParameterAST</code> in the function's parameter list.
<code>`"{}".repeat(param.pointer_depth)`</code>	Reconstruct stars	<code>pointer_depth = 2` → <code>****</code>. <code>"{}".repeat(n)</code> creates a String of <code>n` asterisk characters.</code></code>
<code>`format!("{}", param.c_type, ...)`</code>	Reconstruct type string	<code>"int" + "****" = "int****". The full type string with stars.</code>
<code>`model.symbol_table.insert(param.name.clone(), type_desc)`</code>	Add to dictionary	<code>symbol_table["a"] = "int****". Now we know variable <code>a` is of type <code>int**`.</code></code></code>

Why reconstruct the type string? The AST stores `c_type` and `pointer_depth` separately for easy numeric processing. The symbol table stores them combined (`"int****"`) for human-readable output in reports.

Step 2: Scan Body for Local Variable Declarations (Lines 15-31)

```

let lines: Vec<&str> = func.body_code.lines().collect();

for (idx, &line) in lines.iter().enumerate() {
    let line_num = idx + 1;
    let trimmed = line.trim();

    if (trimmed.starts_with("int ")
        || trimmed.starts_with("char ")
        || trimmed.starts_with("float ")
        || trimmed.starts_with("double ")
        || trimmed.starts_with("void "))
        && trimmed.contains('=')
    {
        let left = trimmed.split('=').next().unwrap().trim();
        let parts: Vec<&str> = left.split_whitespace().collect();
        if parts.len() >= 2 {
            let var_name = parts.last().unwrap().trim_matches('*').to_string();
            let var_type = parts[..parts.len() - 1].join(" ");
            model.symbol_table.insert(var_name, var_type);
        }
    }
}

```

Line	Code	Example with <code>int *temp = *a;</code>
<code>`func.body_code.lines().collect()`</code>	Split body into lines	The <code>body_code</code> string is split at newlines into a <code>Vec</code> .

Line	Code	Example with <code>int *temp = *a;</code>
<code>`lines.iter().enumerate()`</code>	Get (index, line) pairs	<code>`enumerate()`</code> gives both the 0-based index (<code>idx`</code>) and the line content.
<code>`let line_num = idx + 1`</code>	1-based line number	Convert 0-based index to 1-based for human-readable locations.
<code>`trimmed.starts_with("int ")`</code>	Type detection	Looks for local declarations starting with known C types.
<code>`trimmed.contains('=')`</code>	Has assignment	Only process lines that are declarations with initialization.
<code>`trimmed.split('=').next().unwrap()`</code>	Get left side of <code>`=`</code>	<code>`"int *temp = *a" → left side = `int *temp`. <code>`next()`</code> gets the first element of the split iterator. <code>`unwrap()`</code> is safe because <code>`split`</code> always returns at least one element.</code>
<code>`parts.last().unwrap().trim_matches('*')`</code>	Extract variable name	<code>`["int", "*temp"]` → last = <code>`"*temp" → trim <code>`*` → <code>`temp`.</code></code></code></code>
<code>`parts[..parts.len()-1].join("`</code>	Extract type	<code>`["int"]` → join → <code>`"int"`.</code></code>

Step 3: Build Def-Use Chains and Classify Memory Patterns (Lines 33-54)

```

for var_name in model.symbol_table.keys() {
    if trimmed.contains(var_name) {
        // Record usage point
        model.def_use_chains
            .entry(var_name.clone())
            .or_default()
            .push(UsePoint {
                line: line_num,
                column: trimmed.find(var_name).unwrap_or(0) + 1,
            });

        // Classify Memory Access Pattern
        if trimmed.contains(&format!("free({})", var_name))
            || trimmed.contains(&format!("free(*{})", var_name))
        {
            model.memory_patterns.insert(var_name.clone(), MemoryPattern::Free);
        } else if trimmed.contains("malloc(") && trimmed.contains(var_name) {
            model.memory_patterns.insert(var_name.clone(), MemoryPattern::Allocate);
        } else if trimmed.contains(&format!("*{}", var_name)) && trimmed.contains('=') {
            model.memory_patterns.entry(var_name.clone())
                .or_insert(MemoryPattern::Write);
        } else {
            model.memory_patterns.entry(var_name.clone())
                .or_insert(MemoryPattern::Read);
        }
    }
}

```

Def-Use Chain recording:

Code	Explanation
<code>`for var_name in model.symbol_table.keys()``</code>	For every known variable, check if this line uses it.
<code>`if trimmed.contains(var_name)``</code>	Does this line's text contain the variable's name? If yes, it's a "use".
<code>`.entry(var_name.clone()).or_default()``</code>	<code>`.entry()``</code> gets the HashMap entry for the key, or creates it if missing. <code>`.or_default()``</code> inserts a new empty <code>Vec`</code> if there's no entry yet. Returns a mutable reference to the Vec.
<code>`.push(UsePoint { line, column })``</code>	Add this occurrence to the variable's usage list.
<code>`.trimmed.find(var_name).unwrap_or(0) + 1``</code>	Find the column: <code>`.find()``</code> returns the byte offset. <code>`.+1``</code> converts to 1-based column.

Memory Pattern Classification (priority order):

Priority	Pattern	Classification	Example
1st (highest)	<code>`free(var_name)``</code> or <code>`free(*var_name)``</code>	<code>`Free`</code>	<code>`free(ptr)`, `free(*buf)``</code>
2nd	Line contains <code>`malloc(`</code> AND the variable name	<code>`Allocate`</code>	<code>`ptr = malloc(100)``</code>
3rd	<code>`*var_name`</code> + <code>`=`</code> in same line	<code>`Write`</code>	<code>`*ptr = 5`</code> (writing through pointer)
4th (default)	Variable appears on line, no special pattern	<code>`Read`</code>	<code>`printf("%d", x)``</code>

Why `.entry().or_insert()` instead of `.insert()`?

```
// entry + or_insert: only sets if NOT already set (keeps existing value)
model.memory_patterns.entry(var_name.clone()).or_insert(MemoryPattern::Write);

// insert: ALWAYS overwrites the existing value
model.memory_patterns.insert(var_name.clone(), MemoryPattern::Free);
```

Free and Allocate use `.insert()` because they are the MOST important patterns — they override any previously recorded Read or Write pattern for that variable. Write and Read use `.or_insert()` because they should NOT overwrite a previously detected Free or Allocate. The highest-priority classification wins.

The Unit Test (Lines 59-84)

```

#[test]
fn test_base_semantic_analyser() {
    let func = FunctionAST {
        name: "test_fn".to_string(),
        return_type: "void".to_string(),
        parameters: vec![ParameterAST {
            name: "ptr".to_string(),
            c_type: "int".to_string(),
            pointer_depth: 1,
            is_union: false,
        }],
        body_code: "int x = 10;\n*ptr = x;\nfree(ptr);".to_string(),
        call_sites: vec![],
    };

    let model = analyze_function_base(&func);
    assert!(model.symbol_table.contains_key("ptr"));
    assert!(model.symbol_table.contains_key("x"));
    assert!(matches!(model.memory_patterns.get("ptr"),
        Some(MemoryPattern::Free)));
}

```

Assertion	What it verifies
<code>`symbol_table.contains_key("ptr")`</code>	Parameter <code>`ptr`</code> was added to the symbol table from the parameter list.
<code>`symbol_table.contains_key("x")`</code>	Local variable <code>`x`</code> from <code>`int x = 10;`</code> was detected and added.
<code>`memory_patterns.get("ptr") == Some(MemoryPattern::Free)`</code>	<code>`free(ptr)`</code> on line 3 was detected and classified as <code>`Free`</code> — overriding the earlier <code>`Write`</code> from <code>`*ptr = x`</code> .

File 3: `src/module3_semantic/pointer_chain.rs`

(Multi-Level Pointer Depth Analysis)

Purpose: Builds on what `base_analyser` started. While base analysis tracks *WHAT* happens to variables, `pointer_chain` specifically focuses on *HOW DEEP* the pointer indirection goes — because `int*` and `int**` need completely different Rust translations. This sub-analysis handles the precursors to **Idiom 2 (T freeing)** and **Idiom 3 (T allocating)**.

Lines 1-2: Imports

```

use crate::module2_ast::ast_parser::FunctionAST;
use crate::data_stores::semantic_models::{PointerDepthClassification, SemanticModel};

```

- `FunctionAST` — read the parameter list and body code.
- `PointerDepthClassification` — the enum (`SingleLevel` or `MultiLevel { depth }`) we're writing into the model.

Lines 4-40: The Main Function `analyze_pointer_chains()`

```
pub fn analyze_pointer_chains(func: &FunctionAST, model: &mut SemanticModel) {
```

Parameter	Type	Explanation
<code>`func`</code>	<code>`&FunctionAST;`</code>	Read-only reference to the function AST. We only need to read it.
<code>`model`</code>	<code>`&mut; SemanticModel`</code>	MUTABLE reference to the model. We write into <code>`model.pointer_depth_classifications`</code> .

Note: Both parameters are REFERENCES. `analyze_function_base()` owned and returned the model. Now we take a `&mut` reference to that model to ADD more data to it. This is safe in Rust because the `for func in &tu_ast.functions` loop in `mod.rs` ensures we're only analyzing one function at a time.

Phase 1: Classify Parameters by Pointer Depth (Lines 7-21)

```
for param in &func.parameters {
    if param.pointer_depth >= 2 {
        model.pointer_depth_classifications.insert(
            param.name.clone(),
            PointerDepthClassification::MultiLevel {
                depth: param.pointer_depth,
            },
        );
    } else {
        model.pointer_depth_classifications.insert(
            param.name.clone(),
            PointerDepthClassification::SingleLevel,
        );
    }
}
```

Condition	Classification	C Example	Rust Implication
<code>`pointer_depth >= 2`</code>	<code>`MultiLevel { depth }`</code>	<code>`int **a`</code> (depth=2), <code>`int ***p`</code> (depth=3)	Complex ownership — needs Module 4 interprocedural analysis to determine the right Rust type
<code>`pointer_depth < 2`</code>	<code>`SingleLevel`</code>	<code>`int *p`</code> (depth=1) or <code>`int x`</code> (depth=0)	Straightforward: <code>`&T;`</code> , <code>`&mut; T`</code> , or plain value

Why `depth >= 2` is special: In C, `int*` is a simple pointer. But `int**` can mean: - "I'll tell you what value this pointer holds" (output parameter, Idiom 3) - "I'll free the thing this pointer points to" (consume ownership, Idiom 2) - "I'll swap what two pointers point to" (classic swap idiom) Module 4 will use this classification to determine which idiom applies.

Phase 2: Inspect Body for Double-Dereference Patterns (Lines 23-39)

```
for line in func.body_code.lines() {
    let trimmed = line.trim();
    for (var_name, classif) in &model.pointer_depth_classifications {
        if let PointerDepthClassification::MultiLevel { depth: _ } = classif {
            let double_deref = format!("{}", var_name);
            let single_deref = format!("{}", var_name);

            if trimmed.contains(&format!("free({})", single_deref)) {
                // Idiom 2 precursor: Function frees pointee through T**
                model.memory_patterns.insert(
                    var_name.clone(),
                    crate::data_stores::semantic_models::MemoryPattern::Free
                );
            } else if trimmed.contains(&double_deref) && trimmed.contains('=') {
                model.memory_patterns.insert(
                    var_name.clone(),
                    crate::data_stores::semantic_models::MemoryPattern::Write
                );
            }
        }
    }
}
```

Pattern Detected	Code Pattern	Idiom	What it means
<code>`free(*a)`</code>	<code>`free(single_deref)`</code> where <code>`single_deref = "a"`</code>	Idiom 2	Callee frees the heap block the caller's pointer points to. Mark as <code>`Free`</code> .
<code>`**a = value`</code>	<code>`double_deref` + `=`</code>	Idiom 2/swap	Writing through double indirection — modifying the pointer the caller passed. Mark as <code>`Write`</code> .

Pattern string construction:

```
var_name = "a"
single_deref = format!("{}", "a") = "*a"
double_deref = format!("{}", "a") = "**a"

free(*a)    detected by: trimmed.contains("free(*a)")
**a = ..    detected by: trimmed.contains("**a") && trimmed.contains('=')
```

`if let PointerDepthClassification::MultiLevel { depth: _ } = classif:`

- Pattern-matches the classification enum.
- `{ depth: _ }` — we DON'T need the depth number here, just need to confirm it IS multi-level.
- `depth: _` discards the actual depth value (the `_` wildcard).

The Unit Test (Lines 42-78)

```

#[test]
fn test_multi_level_pointer_chain_analyser() {
    let func = FunctionAST {
        name: "swap".to_string(),
        parameters: vec![
            ParameterAST { name: "a".to_string(), c_type: "int".to_string(),
                           pointer_depth: 2, is_union: false },
            ParameterAST { name: "b".to_string(), c_type: "int".to_string(),
                           pointer_depth: 2, is_union: false },
        ],
        body_code: "int *temp = *a;\n*a = *b;\n*b = temp;".to_string(),
        // ...
    };

    let mut model = SemanticModel::new("swap".to_string());
    analyze_pointer_chains(&func, &mut model);

    assert!(matches!(
        model.pointer_depth_classifications.get("a"),
        Some(PointerDepthClassification::MultiLevel { depth: 2 })
    ));
}

```

Line	Explanation
<code>`let mut model = SemanticModel::new(...)`</code>	Creates an empty model. Note: in the real pipeline, <code>`analyze_function_base`</code> creates this. In the test we create it manually.
<code>`analyze_pointer_chains(&func, &mut model)`</code>	Run the function under test, passing a mutable reference.
<code>`Some(PointerDepthClassification::MultiLevel { depth: 2 })`</code>	Pattern match checking that <code>`a`</code> is classified as depth-2 multi-level.

File 4: `src/module3_semantic/union_transformer.rs`

(Converting C Unions to Rust Enum Transforms)

Purpose: C unions are unsafe because multiple fields share the same memory. Rust doesn't allow this pattern. Instead, Rust uses `enum` which is a tagged union — only ONE variant is active at any time, and the type system enforces it. This module determines: for each function that uses a union, CAN we tell locally which field (variant) is being accessed? - If YES: record a resolved `UnionTransform` (we know which variant to generate) - If NO: record an `UnresolvedUnionCase` (kick to Module 4 for cross-function analysis)

Lines 1-2: Imports

```
use crate::module2_ast::ast_parser::{FunctionAST, TranslationUnitAST};
use crate::data_stores::semantic_models::{
    SemanticModel, UnionTransform, UnresolvedUnionCase
};
```

Import	Why needed
`FunctionAST`	The function being analyzed.
`TranslationUnitAST`	The whole program AST — needed to look up union DEFINITIONS (not just usages).
`SemanticModel`	The model we're writing results into.
`UnionTransform`	Stores a successfully resolved union-to-enum mapping.
`UnresolvedUnionCase`	Flags a union that couldn't be resolved locally.

Lines 6-38: The Main Function `transform_unions_per_function()`

```
pub fn transform_unions_per_function(
    func: &FunctionAST,
    tu_ast: &TranslationUnitAST,
    model: &mut SemanticModel,
) {
    for union_def in &tu_ast.unions {
        if func.body_code.contains(&union_def.name)
            || func.parameters.iter().any(|p| p.c_type.contains(&union_def.name))
        {
            // This function accesses this union
        }
    }
}
```

Parameter	Role
`func`	The function we're analyzing.
`tu_ast`	The whole program — we loop through `tu_ast.unions` to find all union definitions.
`model`	Write results into the semantic model.

Union usage detection:

```
func.body_code.contains(&union_def.name)
```

If the union name (e.g., `"Data"`) appears ANYWHERE in the function body, the function uses it.

```
func.parameters.iter().any(|p| p.c_type.contains(&union_def.name))
```

Also check if the union type appears as a PARAMETER. `iter().any(|p| condition)` returns `true` if the condition is true for ANY parameter.

Active Variant Detection (Lines 15-37)

```

let mut resolved_variant = None;

for variant in &union_def.variants {
    if func.body_code.contains(&format!("{}", variant.field_name))
        || func.body_code.contains(&format!("{}", variant.field_name))
    {
        resolved_variant = Some(variant.field_name.clone());
        break;
    }
}

if let Some(variant) = resolved_variant {
    model.local_union_transforms.push(UnionTransform {
        union_name: union_def.name.clone(),
        enum_name: format!("{}", Enum, capitalize(&union_def.name)),
        local_variant_resolved: Some(variant),
    });
} else {
    model.unresolved_union_cases.push(UnresolvedUnionCase {
        union_name: union_def.name.clone(),
        function_name: func.name.clone(),
    });
}

```

Step-by-step field access detection:

Pattern	Code	C Example	Meaning
Direct access	<code>"{field}"</code>	<code>`u.i = 42`</code> — accesses field <code>`i`</code> directly	Local variable: <code>`union Data u; u.i = 42;`</code>
Pointer access	<code>"->{field}"</code>	<code>`u->f = 3.14`</code> — accesses <code>`f`</code> through pointer	Pointer to union: <code>`union Data *u; u->f = 3.14;`</code>

```

format!("{}", variant.field_name) // produces "i"
format!("{}", variant.field_name) // produces "->i"

```

If the active variant IS found locally:

```

model.local_union_transforms.push(UnionTransform {
    union_name: union_def.name.clone(),           // e.g., "Data"
    enum_name: format!("{}", Enum, capitalize(&...)), // e.g., "DataEnum"
    local_variant_resolved: Some(variant),         // e.g., Some("i")
});

```

This tells Module 5: "For this function, when you encounter union `Data`, the active variant is `i`. Generate Rust enum `DataEnum::I(i32)`."

If the active variant is NOT found locally:

```
model.unresolved_union_cases.push(UnresolvedUnionCase {
    union_name: union_def.name.clone(),    // "Data"
    function_name: func.name.clone(),      // "read_val"
});
```

This is a flag for Module 4: "Function `read_val` uses union `Data` but we can't determine which variant is active just by looking at this function. Please check what other functions set before calling this one."

The `capitalize()` Helper (Lines 41-47)

```
fn capitalize(s: &str) -> String {
    let mut c = s.chars();
    match c.next() {
        None => String::new(),
        Some(f) => f.to_uppercase().collect::<String>() + c.as_str(),
    }
}
```

Code	Explanation
<code>`let mut c = s.chars()`</code>	Creates a character iterator over the string. <code>`mut`</code> because <code>`next()`</code> advances it.
<code>`match c.next()`</code>	Gets the FIRST character. Returns <code>`Option`</code> .
<code>`None => String::new()`</code>	Empty string → return empty string.
<code>`Some(f) => f.to_uppercase().collect::()`</code>	Uppercase the first character and collect it into a String. <code>`to_uppercase()`</code> returns an iterator (because some chars uppercase to multiple chars).
<code>`+ c.as_str()`</code>	Concatenate with the REST of the string (all characters after the first). <code>`c.as_str()`</code> returns the remaining characters as a string slice.

Example: `capitalize("data") → "D" + "ata" = "Data"` Then `format!("{}", Enum, "Data") = "DataEnum"`

The Unit Test (Lines 49-83)

```

#[test]
fn test_local_union_transformer() {
    let tu_ast = TranslationUnitAST {
        functions: vec![],
        unions: vec![UnionAST {
            name: "Data".to_string(),
            variants: vec![
                UnionVariantAST { field_name: "i".to_string(),
                                   field_type: "int".to_string() },
                UnionVariantAST { field_name: "f".to_string(),
                                   field_type: "float".to_string() },
            ],
            tag_field: Some("tag".to_string()),
        }],
        structs: vec![],
    };

    let func = FunctionAST {
        name: "process_data".to_string(),
        body_code: "union Data d;\n d.i = 42;".to_string(),
        // ...
    };

    let mut model = SemanticModel::new("process_data".to_string());
    transform_unions_per_function(&func, &tu_ast, &mut model);

    assert_eq!(model.local_union_transforms.len(), 1);
    assert_eq!(
        model.local_union_transforms[0].local_variant_resolved,
        Some("i".to_string())
    );
}

```

Test Setup	Explanation
`tu_ast` with one union `Data`	Define a program-level union with two variants: `i` (int) and `f` (float).
`func` with `body_code: "union Data d;\n d.i = 42;"`	A function that creates a `Data` union and accesses its `i` field.
`assert_eq!(local_union_transforms.len(), 1)`	Exactly ONE resolved transform was created.
`local_variant_resolved == Some("i")`	The `i` field access was correctly detected as the active variant.

How All Three Sub-Modules Work Together

For each FunctionAST in the program:

```
FunctionAST "swap" { parameters: [int **a, int **b], body: "*a = *b; ..." }
    |
    | analyze_function_base(func)
    v
SemanticModel {
    symbol_table: { "a" -> "int**", "b" -> "int**", "temp" -> "int" }
    def_use_chains: { "a" -> [UsePoint{2,1}, UsePoint{3,1}], ... }
    memory_patterns: { "a" -> Write, "b" -> Write, "temp" -> Read }
}
    |
    | analyze_pointer_chains(func, &mut model)
    v
SemanticModel {
    ... (everything above) ...
    pointer_depth_classifications: {
        "a" -> MultiLevel { depth: 2 },
        "b" -> MultiLevel { depth: 2 }
    }
}
    |
    | transform_unions_per_function(func, tu_ast, &mut model)
    v
SemanticModel {
    ... (everything above) ...
    local_union_transforms: [] // "swap" doesn't use any unions
    unresolved_union_cases: []
}
    |
    v
semantic_models.insert("swap", model)
```

Key Rust Concepts Introduced in This Module

	Meaning			
del`	Mutable reference — function can modify the caller's data.			
	Write through a mutable reference.			
y(k).or_defa	Get-or-create pattern for HashMap.			
entry(k).or_in`	Set only if not already set — preserves existing value.			
sert(k,	Always overwrites any existing value.			
cond)`		`func.parameters.iter().any(\	p\	p.c_type.con

	Meaning			
	Build strings dynamically for pattern matching.			
ved_variant`	Unwrap an `Option` safely — only run if it's `Some`.			
	Iterate over Unicode characters (not bytes).			
	Collect an iterator into a specific type, specified with turbofish `::<>`.			
	Exit the loop early once first matching variant found.			
er_chain.rs`	When you haven't added a `use` import, use the full path instead.			

Summary

Module 3 performs **per-function semantic analysis** by running three sequential analyses on each function's `FunctionAST`:

3.1 Base Analyser (`base_analyser.rs`):

- Builds the `symbol_table`: every parameter and local variable → its C type
- Builds `def_use_chains`: every occurrence of every variable, by line and column
- Classifies `memory_patterns`: is each variable Read/Written/Allocated/Freed?
- Priority order: Free > Allocate > Write > Read

3.2 Pointer Chain Analyser (`pointer_chain.rs`):

- Classifies every parameter as `SingleLevel` or `MultiLevel { depth }` in `pointer_depth_classifications`
- For multi-level params, re-checks the function body for double-dereference patterns (`**a`, `free(*a)`)
- Provides the foundation for Idiom 2 and Idiom 3 detection in Module 4

3.3 Union Transformer (`union_transformer.rs`):

- For each union used in the function, tries to detect the active variant by looking for `.field` and `->field` access patterns
- If resolved locally → store a `UnionTransform` with the active variant name
- If not resolvable locally → store an `UnresolvedUnionCase` for Module 4 to handle

The entire output is a `HashMap<String, SemanticModel>` — one model per function — stored as D1 and passed to Module 4 for interprocedural analysis.

Document generated for C2SafeRust-LLM Mini Project S3

Module 3: Per-Function Semantic Analysis — Complete Line-by-Line Explanation

